```
#!/usr/bin/env python3
"""
keraal_parse_annotations.py — KERAAL 데이터셋 Anvil XML 주석 파서 (초안)

목적
----
KERAAL 데이터셋(그룹 1A/2A의 Anvil XML 주석 + 그룹3의 파일명 라벨)을 파싱해
- 녹화 단위(one row per recording) master CSV
- 시간 구간(temporal segment) 단위 long-format CSV
- 오류유형 x 신체부위 교차표
- QC 리포트 (결측치, 클래스 불균형, 참가자 중복 등)
를 생성한다.

전제 / 주의
-----------
* 이 스크립트는 실제 KERAAL zip 파일을 다운로드해서 검증하지 못한 상태로 작성되었다
  (샌드박스 네트워크 정책상 keraal.enstb.org 접근이 막힘, HTTP 403).
  대신 아래 1차 자료를 근거로 구조를 재구성했다:
    - annotate_videos/keraalSpec.xml (Anvil 주석 스펙: 트랙/속성/허용값 정의)
    - annotate_videos/createAnvil.py (실제 주석 XML을 생성하는 스크립트 — 루트 구조 확인용)
    - readme_files_format.txt (Kinect/OpenPose/BlazePose/Vicon 파일 포맷, 관절 순서)
```

```
    - KERAAL 공식 페이지의 파일명 규칙 설명 및
Nguyen et al. IJCNN 2024 논문
  실제 샘플 zip(keraal_sample_2022.zip)을 받는
즉시 --selftest 대신 실제 파일로 돌려서
  아래 "확인 필요" 표시가 붙은 가정들을 검증할 것.

확인이 필요한 가정 (반드시 실제 파일로 재검증)
----------------------------------------
  [A] 파일명 구분자와 순서: 공식 문서마다 대시/언더스코
어 표기가 조금씩 다르다
      (예: "GroupId-Modality-ExerciseName-
SubjectId-RecordingId" vs

"G3_Modality_ExerciseName_ParticipantIdTria
lId_label_RecordingId").
      실제 파일을 받으면 --inspect 로 먼저 목록만
뽑아 정규식을 맞출 것.
  [B] Group3 파일명의 바디파트 인코딩: 공식 페이지
예시는 "Error1Bodypart1"처럼
      오류유형과 신체부위 인덱스가 합쳐진 형태만 보여준
다. 인덱스(1~4)가
      LeftArm/RightArm/BothArms/Spine 중 무엇
에 대응하는지는 문서에 없으므로,
      LabellisationDesVideos_fr.pdf(프랑스어
라벨 설명)를 직접 대조해야 한다.
      이 스크립트는 group3 라벨 문자열을 있는 그대로
보존하고, 느슨한 정규식으로
      ErrorN / BodypartM을 뽑아내되 "raw" 컬럼
도 항상 함께 남긴다.
  [C] "Temporal error" 트랙의 <el> 개수: 한 녹
화에 오류 구간이 여러 개(복수 <el>)
      있을 수 있다고 보고 long-format으로 모두 펼친
다. 실제로 몇 개가 나오는지는
      확인 필요.
```

```
  [D] bodyPart/Evaluation(중요도) 속성이 항상 채
워져 있는지: keraalSpec.xml에는
      Global error/Temporal error 트랙의
Evaluation, bodyPart 속성에
      emptyvalue/defaultvalue가 명시돼 있지 않
다. 즉 의사가 값을 비워둘 수 있다.
      이 스크립트는 결측을 "MISSING"으로 명시적으로
표기하고 QC 리포트에 개수를 낸다.

사용법
------
  # 1) 먼저 파일명 패턴이 맞는지만 확인 (파싱 없이 목
록만 출력)
  python keraal_parse_annotations.py
inspect --root /path/to/keraal

  # 2) 실제 파싱 실행
  python keraal_parse_annotations.py parse
--root /path/to/keraal --outdir
./keraal_parsed

  # 3) 합성 데이터로 자체 테스트 (실제 데이터 없이 스
크립트 동작만 검증)
  python keraal_parse_annotations.py
selftest --outdir ./keraal_selftest

디렉터리 구조 가정 (공식 배포 구조 기준)
--------------------------------------
  <root>/
    group1A/
      annotations/*.xml      <- Anvil XML
(Global evaluation, Global error, Temporal
error 트랙)
      kinect/*.txt
```

```
      videos/*.mp4
    group2A/
      annotations/*.xml
      kinect/*.txt
    group3/
      kinect/*.txt            <- 파일명 자체에
라벨 포함, XML 주석 없음
      videos/*.avi
    group1B/, group2B/        <- 라벨 없음 (선
택적으로 목록만 수집)

의존성
------
표준 라이브러리 + defusedxml (있으면 사용, 없으면
xml.etree로 폴백; 신뢰되지 않는
XML을 열 가능성을 감안해 defusedxml을 우선한다).
"""

from __future__ import annotations

import argparse
import csv
import json
import re
import sys
from collections import Counter,
defaultdict
from dataclasses import dataclass, field,
asdict
from pathlib import Path
from typing import Optional

try:
    from defusedxml import ElementTree as
```

```
ET  # type: ignore
except ImportError:  # pragma: no cover -
fallback only
    import xml.etree.ElementTree as ET  #
nosec - fallback if defusedxml unavailable


# ----------------------------------------
--------------------------------
# 허용값 (keraalSpec.xml 기준) — QC에서 스펙 밖
값이 나오면 경고
# ----------------------------------------
--------------------------------

VALID_EVALUATION = {"Correct", "Incorrect",
"Incomplete", "Motionless"}
VALID_ERROR_TYPE = {"none", "Error1",
"Error2", "Error3", "Other"}
VALID_SEVERITY = {"SmallError",
"LargeError"}
VALID_BODYPART = {"LeftArm", "RightArm",
"BothArms", "Spine"}
MISSING = "MISSING"

EXERCISE_CODES = {"RTK": "torso_rotation",
"ELK": "flank_stretch", "CTK":
"hiding_face"}


# ----------------------------------------
--------------------------------
# 데이터 구조
# ----------------------------------------
--------------------------------
```

```
@dataclass
class TemporalSegment:
    recording_id: str
    start: str
    end: str
    error_type: str
    severity: str
    body_part: str


@dataclass
class RecordingRecord:
    recording_id: str          # 우리가 부여하
는 고유 키 (source_file 기반)
    group: str                 # 1A / 2A /
3 / 1B / 2B
    exercise_code: str         # RTK / ELK
/ CTK / UNKNOWN
    exercise_name: str
    participant_id: str
    trial_id: str              # group3만 해
당, 그 외는 ""
    source_file: str           # 원본 파일 경
로 (문자열)
    label_source: str          # "xml" |
"filename" | "none"
    global_evaluation: str
    global_error_type: str
    global_severity: str
    global_body_part: str
    n_temporal_segments: int
    raw_group3_label: str = ""  # group3 파
일명에서 추출한 원본 라벨 문자열
```

```
    parse_warnings: str = ""


# ----------------------------------------
--------------------------------
# 파일명 파싱 (그룹 1A/1B/2A/2B, 그룹 3)
# ----------------------------------------
--------------------------------

# 확인 필요 [A]: 실제 파일을 받으면 이 정규식들을 먼저
--inspect 로 검증할 것.
# 구분자를 '-' 또는 '_' 둘 다 허용하도록 느슨하게 설계
했다.
_SEP = r"[-_]"

RE_GROUP_1_2 = re.compile(
    rf"G(?P<group>1A|1B|2A|2B){_SEP}(?
P<modality>\w+){_SEP}"
    rf"(?P<exercise>RTK|ELK|CTK){_SEP}(?
P<participant>[A-Za-z0-9]+){_SEP}"
    rf"(?P<recording>[A-Za-z0-9]+)",
    re.IGNORECASE,
)

RE_GROUP_3 = re.compile(
    rf"G3{_SEP}(?:(?P<modality>\w+){_SEP})?
(?P<exercise>RTK|ELK|CTK){_SEP}"
    rf"(?P<participant>[A-Za-z]+\d+?)(?
P<trial>T\d+)?{_SEP}"
    rf"(?P<label>[A-Za-z0-9]+){_SEP}(?
P<recording>[A-Za-z0-9]+)",
    re.IGNORECASE,
)
```

```
# group3 라벨 문자열에서 ErrorN / BodypartM 을
느슨하게 추출 (확인 필요 [B])
RE_G3_LABEL_PARTS = re.compile(r"
(Error\d+|Correct)", re.IGNORECASE)
RE_G3_BODYPART =
re.compile(r"Bodypart(\d+)", re.IGNORECASE)


def parse_filename_group1_2(path: Path) ->
Optional[dict]:
    m = RE_GROUP_1_2.search(path.stem)
    if not m:
        return None
    d = m.groupdict()
    return {
        "group": d["group"].upper(),
        "exercise_code":
d["exercise"].upper(),
        "participant_id": d["participant"],
        "recording_id_raw": d["recording"],
    }


def parse_filename_group3(path: Path) ->
Optional[dict]:
    m = RE_GROUP_3.search(path.stem)
    if not m:
        return None
    d = m.groupdict()
    return {
        "group": "3",
        "exercise_code":
d["exercise"].upper(),
        "participant_id": d["participant"],
```

```
        "trial_id": d.get("trial") or "",
        "raw_label": d["label"],
        "recording_id_raw": d["recording"],
    }


def decode_group3_label(raw_label: str) ->
dict:
    """Group3 파일명 라벨(예:
'Error1Bodypart1', 'Correct')을 최대한 분해한
다.
    확인 필요 [B]: Bodypart 인덱스 ->
VALID_BODYPART 매핑은 아직 검증되지 않았다.
    검증 전까지는 인덱스를 그대로 두고 'raw'만 신뢰한
다.
    """
    out = {
        "global_evaluation": MISSING,
        "global_error_type": MISSING,
        "global_body_part": MISSING,
        "warning": "",
    }
    if raw_label.strip().lower() ==
"correct":
        out["global_evaluation"] =
"Correct"
        out["global_error_type"] = "none"
        return out

    err_m =
RE_G3_LABEL_PARTS.search(raw_label)
    bp_m = RE_G3_BODYPART.search(raw_label)
    if err_m:
        out["global_evaluation"] =
```

```
"Incorrect"
        out["global_error_type"] =
err_m.group(1).capitalize()
    else:
        out["warning"] = f"라벨에서 오류유형을
추출하지 못함: '{raw_label}'"

    if bp_m:
        # 인덱스만 기록하고 실제 이름 매핑은 미확정 상
태로 남김 (확인 필요 [B])
        out["global_body_part"] =
f"BodypartIndex{bp_m.group(1)}(매핑 미확인)"
    return out


# ----------------------------------------
---------------------------------
# Anvil XML 파싱 (그룹 1A / 2A)
# ----------------------------------------
---------------------------------

def _attr_value(el, name: str) -> str:
    """<el> 안의 <attribute
name="...">value</attribute>를 찾아 텍스트 반
환.

    속성 태그 자체가 없거나(예: 이 <el>에 bodyPart
속성이 아예 없음),
    있지만 내용이 빈 문자열인 경우(예: <attribute
name="bodyPart"></attribute>,
    의사가 값을 비워둔 경우) 모두 MISSING으로 통일한
다. 두 경우를 구분해야 할
    필요가 생기면 반환값을 (value, found_tag:
bool) 튜플로 바꿀 것.
```

```
    """
    for attr in el.findall("attribute"):
        if attr.get("name") == name:
            text = (attr.text or
"").strip()
            return text if text else
MISSING
    return MISSING


def parse_anvil_xml(xml_path: Path) ->
dict:
    """Anvil XML 한 개를 파싱해 global 평가/오류
+ temporal 구간 리스트를 반환.

    createAnvil.py 기준 구조:
      <annotation>
        <head>...<video src="..."/></head>
        <body>
          <track name="Global evaluation">
            <el index="0" start="0"
end="...">
              <attribute
name="evaluation">Correct</attribute>
            </el>
          </track>
          <track name="Global error">
            <el index="0" start="0"
end="...">
              <attribute
name="type">Error1</attribute>
              <attribute
name="Evaluation">SmallError</attribute>
(확인 필요 [D])
```

```
              <attribute
name="bodyPart">LeftArm</attribute>
(확인 필요 [D])
            </el>
          </track>
          <track name="Temporal error">
            <el index="0" start="12.3"
end="18.9">
              <attribute
name="type">Error1</attribute>
              <attribute
name="Evaluation">SmallError</attribute>
              <attribute
name="bodyPart">LeftArm</attribute>
            </el>
            ... (확인 필요 [C]: 여러 개 존재 가
능) ...
          </track>
        </body>
      </annotation>
    """
    result = {
        "global_evaluation": MISSING,
        "global_error_type": MISSING,
        "global_severity": MISSING,
        "global_body_part": MISSING,
        "temporal_segments": [],  #
list[dict(start,end,error_type,severity,bod
y_part)]
        "warnings": [],
    }

    try:
        tree = ET.parse(str(xml_path))
```

```
    except Exception as exc:  # noqa:
BLE001 - 파싱 실패는 QC로 넘긴다
        result["warnings"].append(f"XML 파싱
실패: {exc}")
        return result

    root = tree.getroot()
    body = root.find("body")
    if body is None:
        result["warnings"].append("<body>
태그를 찾지 못함")
        return result

    for track in body.findall("track"):
        track_name = track.get("name", "")
        els = track.findall("el")

        if track_name == "Global
evaluation":
            if not els:

result["warnings"].append("Global
evaluation 트랙에 <el> 없음")
            for el in els:
                val = _attr_value(el,
"evaluation")
                result["global_evaluation"]
= val
                if val not in
VALID_EVALUATION and val != MISSING:

result["warnings"].append(f"스펙 밖
evaluation 값: '{val}'")
```

```
        elif track_name == "Global error":
            if not els:

result["warnings"].append("Global error 트랙
에 <el> 없음")
            for el in els:
                etype = _attr_value(el,
"type")
                sev = _attr_value(el,
"Evaluation")
                bpart = _attr_value(el,
"bodyPart")
                result["global_error_type"]
= etype
                result["global_severity"] =
sev
                result["global_body_part"]
= bpart
                if etype not in
VALID_ERROR_TYPE and etype != MISSING:

result["warnings"].append(f"스펙 밖 error
type 값: '{etype}'")
                if sev not in
VALID_SEVERITY and sev != MISSING:

result["warnings"].append(f"스펙 밖 severity
값: '{sev}'")
                if bpart not in
VALID_BODYPART and bpart != MISSING:

result["warnings"].append(f"스펙 밖 bodyPart
값: '{bpart}'")
```

```
        elif track_name == "Temporal
error":
            for el in els:
                seg = {
                    "start":
el.get("start", ""),
                    "end": el.get("end",
""),
                    "error_type":
_attr_value(el, "type"),
                    "severity":
_attr_value(el, "Evaluation"),
                    "body_part":
_attr_value(el, "bodyPart"),
                }

result["temporal_segments"].append(seg)

    return result


# ----------------------------------------
--------------------------------
# 그룹별 수집 로직
# ----------------------------------------
--------------------------------

def collect_group_1_2(root: Path, group:
str, warnings_out: list[str]) ->
list[RecordingRecord]:
    """그룹 1A/2A (XML 주석 있음) 또는 1B/2B (주
석 없음, kinect 파일만) 수집."""
    group_dir = root / f"group{group}"
    records: list[RecordingRecord] = []
```

```
    ann_dir = group_dir / "annotations"
    kinect_dir = group_dir / "kinect"

    xml_files =
sorted(ann_dir.glob("*.xml")) if
ann_dir.exists() else []
    kinect_files =
sorted(kinect_dir.glob("*.txt")) if
kinect_dir.exists() else []

    if not xml_files and not kinect_files:
        warnings_out.append(f"
[group{group}] annotations/ 또는 kinect/ 폴더
를 찾지 못함: {group_dir}")
        return records

    # XML이 있는 경우: XML을 기준으로, 같은 이름의
kinect 파일을 매칭 시도
    if xml_files:
        for xml_path in xml_files:
            meta =
parse_filename_group1_2(xml_path)
            if meta is None:
                warnings_out.append(f"
[group{group}] 파일명 패턴 불일치 (정규식 확인 필요
[A]): {xml_path.name}")
                continue

            parsed =
parse_anvil_xml(xml_path)
            exercise_code =
meta["exercise_code"]
            rec = RecordingRecord(
```

```
                recording_id=xml_path.stem,
                group=group,

exercise_code=exercise_code,

exercise_name=EXERCISE_CODES.get(exercise_c
ode, "UNKNOWN"),

participant_id=meta["participant_id"],
                trial_id="",
                source_file=str(xml_path),
                label_source="xml",

global_evaluation=parsed["global_evaluation
"],

global_error_type=parsed["global_error_type
"],

global_severity=parsed["global_severity"],

global_body_part=parsed["global_body_part"]
,

n_temporal_segments=len(parsed["temporal_se
gments"]),
                parse_warnings="
".join(parsed["warnings"]),
            )
            records.append(rec)

            for seg in
parsed["temporal_segments"]:
                TEMPORAL_SEGMENTS.append(
```

```
                    TemporalSegment(
recording_id=rec.recording_id,
                        start=seg["start"],
                        end=seg["end"],
error_type=seg["error_type"],
severity=seg["severity"],
body_part=seg["body_part"],
                    )
                )
    else:
        # 1B/2B: 라벨 없이 존재 목록만 수집 (향후
semi-supervised 용도)
        for kfile in kinect_files:
            meta =
parse_filename_group1_2(kfile)
            if meta is None:
                warnings_out.append(f"
[group{group}] 파일명 패턴 불일치 (정규식 확인 필요
[A]): {kfile.name}")
                continue
            exercise_code =
meta["exercise_code"]
            records.append(
                RecordingRecord(
recording_id=kfile.stem,
                    group=group,
exercise_code=exercise_code,
```

```
exercise_name=EXERCISE_CODES.get(exercise_c
ode, "UNKNOWN"),

participant_id=meta["participant_id"],
                    trial_id="",
                    source_file=str(kfile),
                    label_source="none",

global_evaluation=MISSING,

global_error_type=MISSING,

global_severity=MISSING,

global_body_part=MISSING,
                    n_temporal_segments=0,
                )
            )

    return records


def collect_group_3(root: Path,
warnings_out: list[str]) ->
list[RecordingRecord]:
    """그룹3: XML 주석이 없고, 파일명 자체가 라벨이
다 (확인 필요 [B])."""
    group_dir = root / "group3"
    kinect_dir = group_dir / "kinect"
    records: list[RecordingRecord] = []

    if not kinect_dir.exists():
        warnings_out.append(f"[group3]
kinect/ 폴더를 찾지 못함: {group_dir}")
```

```
        return records

    for kfile in
sorted(kinect_dir.glob("*.txt")):
        meta = parse_filename_group3(kfile)
        if meta is None:
            warnings_out.append(f"[group3]
파일명 패턴 불일치 (정규식 확인 필요 [A]):
{kfile.name}")
            continue

        decoded =
decode_group3_label(meta["raw_label"])
        exercise_code =
meta["exercise_code"]
        rec = RecordingRecord(
            recording_id=kfile.stem,
            group="3",
            exercise_code=exercise_code,

exercise_name=EXERCISE_CODES.get(exercise_c
ode, "UNKNOWN"),

participant_id=meta["participant_id"],
            trial_id=meta.get("trial_id",
""),
            source_file=str(kfile),
            label_source="filename",

global_evaluation=decoded["global_evaluatio
n"],

global_error_type=decoded["global_error_typ
e"],
```

```
            global_severity=MISSING,  #
group3 파일명에는 severity 정보 없음 (확인 필요)

global_body_part=decoded["global_body_part"
],
            n_temporal_segments=0,

raw_group3_label=meta["raw_label"],

parse_warnings=decoded["warning"],
        )
        records.append(rec)

    return records


# 모듈 레벨 버퍼 (간단함을 위해; 큰 규모로 갈 경우 클래
스로 리팩터링 권장)
TEMPORAL_SEGMENTS: list[TemporalSegment] =
[]


# ----------------------------------------
--------------------------------
# 출력 (CSV / 교차표 / QC 리포트)
# ----------------------------------------
--------------------------------

def write_master_csv(records:
list[RecordingRecord], out_path: Path) ->
None:
    fields =
list(asdict(records[0]).keys()) if records
else [f.name for f in
```

```
RecordingRecord.__dataclass_fields__.values
()]
    with out_path.open("w", newline="",
encoding="utf-8-sig") as f:
        writer = csv.DictWriter(f,
fieldnames=fields)
        writer.writeheader()
        for r in records:
            writer.writerow(asdict(r))


def write_temporal_csv(segments:
list[TemporalSegment], out_path: Path) ->
None:
    fields = ["recording_id", "start",
"end", "error_type", "severity",
"body_part"]
    with out_path.open("w", newline="",
encoding="utf-8-sig") as f:
        writer = csv.DictWriter(f,
fieldnames=fields)
        writer.writeheader()
        for s in segments:
            writer.writerow(asdict(s))


def write_crosstab(records:
list[RecordingRecord], out_path: Path) ->
None:
    """오류유형 x 신체부위 교차표 (전역 라벨 기준,
XML 주석 있는 그룹1A/2A만)."""
    counter: Counter[tuple[str, str]] =
Counter()
    for r in records:
```

```
        if r.label_source != "xml":
            continue
        if r.global_error_type in ("none",
MISSING):
            continue
        counter[(r.global_error_type,
r.global_body_part)] += 1

    body_parts = sorted(VALID_BODYPART |
{MISSING})
    error_types = sorted({et for (et, _bp)
in counter} | {"Error1", "Error2",
"Error3", "Other"})

    with out_path.open("w", newline="",
encoding="utf-8-sig") as f:
        writer = csv.writer(f)
        writer.writerow(["error_type"] +
body_parts)
        for et in error_types:
            row = [et] + [counter.get((et,
bp), 0) for bp in body_parts]
            writer.writerow(row)


def build_qc_report(records:
list[RecordingRecord], segments:
list[TemporalSegment],
                    collection_warnings:
list[str]) -> str:
    lines: list[str] = []
    lines.append("=" * 70)
    lines.append("KERAAL 주석 파싱 QC 리포트")
    lines.append("=" * 70)
```

```
    total = len(records)
    lines.append(f"\n총 녹화 수: {total}")

    by_group = Counter(r.group for r in
records)
    lines.append("\n[그룹별 녹화 수]")
    for g, c in sorted(by_group.items()):
        lines.append(f"  group{g}: {c}")

    by_group_exercise = Counter((r.group,
r.exercise_name) for r in records)
    lines.append("\n[그룹 x 운동별 녹화 수]")
    for (g, ex), c in
sorted(by_group_exercise.items()):
        lines.append(f"  group{g} / {ex}:
{c}")

    labeled = [r for r in records if
r.label_source in ("xml", "filename")]
    lines.append(f"\n라벨이 있는 녹화 수:
{len(labeled)} / {total}")

    by_eval = Counter(r.global_evaluation
for r in labeled)
    lines.append("\n[전역 평가(Global
evaluation) 분포]")
    for k, c in sorted(by_eval.items()):
        lines.append(f"  {k}: {c}")

    by_error = Counter(r.global_error_type
for r in labeled)
    lines.append("\n[전역 오류유형 분포]")
    for k, c in sorted(by_error.items()):
```

```
        lines.append(f"  {k}: {c}")

    by_bp = Counter(r.global_body_part for
r in labeled)
    lines.append("\n[전역 신체부위 분포]")
    for k, c in sorted(by_bp.items()):
        lines.append(f"  {k}: {c}")

    missing_bp = [r for r in labeled if
r.global_error_type not in ("none",
MISSING)
                  and r.global_body_part ==
MISSING]
    lines.append(f"\n오류가 있는데 신체부위가 결측
인 녹화 수: {len(missing_bp)}")
    if missing_bp:
        lines.append("  예시(최대 5개): " + ",
".join(r.recording_id for r in
missing_bp[:5]))

    # 참가자 중복 확인 (같은 participant_id가 서로
다른 group에 걸쳐 있으면 subject leakage 위험)
    participants_by_group: dict[str,
set[str]] = defaultdict(set)
    for r in records:

participants_by_group[r.group].add(r.partic
ipant_id)
    lines.append("\n[그룹별 고유 참가자 수 (참가자
ID 매핑 방식에 따라 다를 수 있음)]")
    for g, pset in
sorted(participants_by_group.items()):
        lines.append(f"  group{g}:
{len(pset)}명 -> {sorted(pset)}")
```

```
    overlap_1a_1b =
participants_by_group.get("1A", set()) &
participants_by_group.get("1B", set())
    if overlap_1a_1b:
        lines.append(f"\n참고: group1A/1B 참
가자 ID가 겹침 (같은 환자의 주석/비주석 데이터로 정상):
{sorted(overlap_1a_1b)}")

    n_temporal = len(segments)
    lines.append(f"\n총 temporal segment 수:
{n_temporal} (확인 필요 [C])")

    g3_warn = [r for r in records if
r.group == "3" and r.parse_warnings]
    if g3_warn:
        lines.append(f"\nGroup3 라벨 해석 경고
{len(g3_warn)}건 (확인 필요 [B]):")
        for r in g3_warn[:10]:
            lines.append(f"
{r.recording_id}:
raw='{r.raw_group3_label}'
warning='{r.parse_warnings}'")

    all_parse_warnings = [r for r in
records if r.parse_warnings and r.group !=
"3"]
    if all_parse_warnings:
        lines.append(f"\nXML/스펙 관련 경고
{len(all_parse_warnings)}건:")
        for r in all_parse_warnings[:10]:
            lines.append(f"
{r.recording_id}: {r.parse_warnings}")
```

```
    if collection_warnings:
        lines.append(f"\n[파일 수집 단계 경고
{len(collection_warnings)}건]")
        for w in collection_warnings[:20]:
            lines.append(f"  - {w}")

    lines.append("\n" + "=" * 70)
    lines.append("다음 확인 필요 항목 요약
(README/스크립트 상단 docstring 참고)")
    lines.append("  [A] 파일명 정규식이 실제 파일
과 맞는지  -> inspect 서브커맨드로 먼저 확인")
    lines.append("  [B] Group3
BodypartIndex -> 실제 신체부위 매핑 확정 필요")
    lines.append("  [C] Temporal error 트랙의
실제 <el> 개수/형태 확인")
    lines.append("  [D] bodyPart/Evaluation
결측 비율이 실제로 얼마나 되는지 확인")
    lines.append("=" * 70)

    return "\n".join(lines)


# ----------------------------------------
--------------------------------
# inspect 서브커맨드: 파싱 전에 파일명이 정규식과 맞는
지만 빠르게 확인
# ----------------------------------------
--------------------------------

def cmd_inspect(root: Path) -> None:
    print(f"루트 디렉터리: {root}\n")
    for group in ["1A", "1B", "2A", "2B"]:
        group_dir = root / f"group{group}"
        if not group_dir.exists():
```

```
            print(f"[group{group}] 폴더 없음:
{group_dir}")
            continue
        ann_dir = group_dir / "annotations"
        kinect_dir = group_dir / "kinect"
        xml_files =
sorted(ann_dir.glob("*.xml")) if
ann_dir.exists() else []
        kinect_files =
sorted(kinect_dir.glob("*.txt")) if
kinect_dir.exists() else []

        print(f"[group{group}] annotations:
{len(xml_files)}개, kinect:
{len(kinect_files)}개")
        sample_files = xml_files[:3] or
kinect_files[:3]
        for f in sample_files:
            meta =
parse_filename_group1_2(f)
            status = "OK" if meta else "패턴
불일치!"
            print(f"    예시: {f.name}  ->
{status}  {meta}")
        print()

    group3_dir = root / "group3" / "kinect"
    if group3_dir.exists():
        g3_files =
sorted(group3_dir.glob("*.txt"))
        print(f"[group3] kinect:
{len(g3_files)}개")
        for f in g3_files[:5]:
            meta = parse_filename_group3(f)
```

```
            status = "OK" if meta else "패턴
불일치!"
            decoded =
decode_group3_label(meta["raw_label"]) if
meta else {}
            print(f"    예시: {f.name}  ->
{status}  {meta}  decoded={decoded}")
    else:
        print(f"[group3] 폴더 없음: {root /
'group3'}")


# ----------------------------------------
--------------------------------
# parse 서브커맨드: 실제 파싱 실행
# ----------------------------------------
--------------------------------

def cmd_parse(root: Path, outdir: Path) ->
None:
    global TEMPORAL_SEGMENTS
    TEMPORAL_SEGMENTS = []

    outdir.mkdir(parents=True,
exist_ok=True)
    collection_warnings: list[str] = []

    all_records: list[RecordingRecord] = []
    for group in ["1A", "1B", "2A", "2B"]:

all_records.extend(collect_group_1_2(root,
group, collection_warnings))

all_records.extend(collect_group_3(root,
```

```
collection_warnings))

    if not all_records:
        print("경고: 파싱된 녹화가 하나도 없습니다.
--root 경로와 폴더 구조를 확인하세요.",
              file=sys.stderr)

    master_path = outdir /
"master_annotations.csv"
    temporal_path = outdir /
"temporal_segments_long.csv"
    crosstab_path = outdir /
"crosstab_error_bodypart.csv"
    qc_path = outdir / "qc_report.txt"

    if all_records:
        write_master_csv(all_records,
master_path)
    write_temporal_csv(TEMPORAL_SEGMENTS,
temporal_path)
    write_crosstab(all_records,
crosstab_path)

    qc_text = build_qc_report(all_records,
TEMPORAL_SEGMENTS, collection_warnings)
    qc_path.write_text(qc_text,
encoding="utf-8")

    print(qc_text)
    print(f"\n출력 파일:\n  {master_path}\n
{temporal_path}\n  {crosstab_path}\n
{qc_path}")
```

```
# ----------------------------------------
--------------------------------
# selftest 서브커맨드: 합성 데이터로 스크립트 자체 검
증 (실제 데이터 불필요)
# ----------------------------------------
--------------------------------

_ANVIL_TEMPLATE = """<?xml version="1.0"
encoding="UTF-8"?>
<annotation>
  <head>
    <specification src="../keraalSpec.xml"
/>
    <video src="{video}" master="true" />
  </head>
  <body>
    <track name="Global evaluation">
      <el index="0" start="0" end="{dur}">
        <attribute name="evaluation">
{evaluation}</attribute>
      </el>
    </track>
    <track name="Global error">
      <el index="0" start="0" end="{dur}">
        <attribute name="type">{error_type}
</attribute>
        <attribute name="Evaluation">
{severity}</attribute>
        <attribute name="bodyPart">
{body_part}</attribute>
      </el>
    </track>
    <track name="Temporal error">
{temporal_els}
```

```
    </track>
  </body>
</annotation>
"""

_TEMPORAL_EL_TEMPLATE = """      <el
index="{idx}" start="{start}" end="{end}">
        <attribute name="type">{error_type}
</attribute>
        <attribute name="Evaluation">
{severity}</attribute>
        <attribute name="bodyPart">
{body_part}</attribute>
      </el>"""


def _make_synthetic_xml(path: Path,
evaluation: str, error_type: str, severity:
str,
                        body_part: str,
temporal: list[dict], missing_bp: bool =
False) -> None:
    temporal_els = "\n".join(
        _TEMPORAL_EL_TEMPLATE.format(
            idx=i, start=seg["start"],
end=seg["end"],
            error_type=seg["error_type"],
severity=seg["severity"],
            body_part=seg["body_part"],
        )
        for i, seg in enumerate(temporal)
    )
    xml_text = _ANVIL_TEMPLATE.format(
        video=path.stem + ".mp4",
```

```
        dur="30.0",
        evaluation=evaluation,
        error_type=error_type,
        severity=severity,
        body_part="" if missing_bp else
body_part,
        temporal_els=temporal_els,
    )
    path.write_text(xml_text,
encoding="utf-8")


def cmd_selftest(outdir: Path) -> None:
    root = outdir / "synthetic_dataset"
    for group in ["1A", "2A"]:
        (root / f"group{group}" /
"annotations").mkdir(parents=True,
exist_ok=True)
        (root / f"group{group}" /
"kinect").mkdir(parents=True,
exist_ok=True)
    (root / "group1B" /
"kinect").mkdir(parents=True,
exist_ok=True)
    (root / "group2B" /
"kinect").mkdir(parents=True,
exist_ok=True)
    (root / "group3" /
"kinect").mkdir(parents=True,
exist_ok=True)

    # group1A: 환자 2명, 3개 운동, 정상/오류 섞어
서
    cases = [
```

```
        ("G1A_Kinect_RTK_P01_R001",
"Correct", "none", MISSING, MISSING, []),
        ("G1A_Kinect_RTK_P01_R002",
"Incorrect", "Error1", "SmallError",
"BothArms",
         [{"start": "2.0", "end": "5.5",
"error_type": "Error1", "severity":
"SmallError", "body_part": "BothArms"}]),
        ("G1A_Kinect_ELK_P02_R001",
"Incorrect", "Error2", "LargeError",
"Spine",
         [{"start": "1.0", "end": "9.0",
"error_type": "Error2", "severity":
"LargeError", "body_part": "Spine"}]),
        # 결측 bodyPart 케이스 (확인 필요 [D] 재
현)
        ("G1A_Kinect_CTK_P02_R002",
"Incorrect", "Error3", "SmallError",
MISSING, []),
    ]
    for stem, ev, et, sev, bp, temporal in
cases:
        xml_path = root / "group1A" /
"annotations" / f"{stem}.xml"
        _make_synthetic_xml(xml_path, ev,
et, sev, bp, temporal, missing_bp=(bp ==
MISSING))
        (root / "group1A" / "kinect" / f"
{stem}.txt").write_text("0.0 0.0 0.0\n",
encoding="utf-8")

    # group2A: 건강인 1명
    cases2 = [
        ("G2A_Kinect_RTK_H01_R001",
```

```
"Correct", "none", MISSING, MISSING, []),
        ("G2A_Kinect_ELK_H01_R002",
"Incorrect", "Error1", "SmallError",
"LeftArm",
         [{"start": "0.5", "end": "3.0",
"error_type": "Error1", "severity":
"SmallError", "body_part": "LeftArm"},
          {"start": "10.0", "end": "12.0",
"error_type": "Error1", "severity":
"SmallError", "body_part": "LeftArm"}]),
    ]
    for stem, ev, et, sev, bp, temporal in
cases2:
        xml_path = root / "group2A" /
"annotations" / f"{stem}.xml"
        _make_synthetic_xml(xml_path, ev,
et, sev, bp, temporal)
        (root / "group2A" / "kinect" / f"
{stem}.txt").write_text("0.0 0.0 0.0\n",
encoding="utf-8")

    # group1B / group2B: 라벨 없는 kinect 파일
만
    (root / "group1B" / "kinect" /
"G1B_Kinect_RTK_P01_R010.txt").write_text("
0.0\n", encoding="utf-8")
    (root / "group2B" / "kinect" /
"G2B_Kinect_ELK_H01_R010.txt").write_text("
0.0\n", encoding="utf-8")

    # group3: 파일명 자체가 라벨 (공식 예시
'Error1Bodypart1' 패턴 + Correct 케이스)
    g3_files = [
```

```
"G3_Kinect_RTK_S01T1_Correct_001.txt",

"G3_Kinect_RTK_S01T2_Error1Bodypart1_002.tx
t",

"G3_Kinect_ELK_S02T1_Error2Bodypart3_003.tx
t",

"G3_Kinect_CTK_S02T2_WeirdLabelXYZ_004.txt"
,  # 파싱 실패 유도 (경고 확인용)
    ]
    for name in g3_files:
        (root / "group3" / "kinect" /
name).write_text("0.0\n", encoding="utf-8")

    print(">>> 1) inspect 실행 결과\n")
    cmd_inspect(root)

    print("\n>>> 2) parse 실행 결과\n")
    cmd_parse(root, outdir /
"parsed_output")

    print("\n합성 데이터셋 위치:", root)
    print("파싱 결과 위치:", outdir /
"parsed_output")


# ----------------------------------------
---------------------------------
# CLI
# ----------------------------------------
---------------------------------

def main() -> None:
```

```
    parser =
argparse.ArgumentParser(description=__doc__
,
formatter_class=argparse.RawDescriptionHelp
Formatter)
    sub =
parser.add_subparsers(dest="command",
required=True)

    p_inspect = sub.add_parser("inspect",
help="파일명 패턴이 실제 데이터와 맞는지 미리보기")
    p_inspect.add_argument("--root",
type=Path, required=True, help="KERAAL 데이터
셋 루트 디렉터리")

    p_parse = sub.add_parser("parse",
help="실제 파싱 실행")
    p_parse.add_argument("--root",
type=Path, required=True, help="KERAAL 데이터
셋 루트 디렉터리")
    p_parse.add_argument("--outdir",
type=Path, required=True, help="결과 CSV/리포
트 출력 폴더")

    p_selftest = sub.add_parser("selftest",
help="합성 데이터로 스크립트 자체 검증")
    p_selftest.add_argument("--outdir",
type=Path,
default=Path("./keraal_selftest"))

    args = parser.parse_args()

    if args.command == "inspect":
        cmd_inspect(args.root)
```

```
    elif args.command == "parse":
        cmd_parse(args.root, args.outdir)
    elif args.command == "selftest":
        cmd_selftest(args.outdir)


if __name__ == "__main__":
    main()
```